A
Project Report On

Associative Rule Mining of Words
(Word Prediction)

By:
Kushal Shah (CE142)(19CEUOS160)
Chintan Solanki (CE148)(19CEUBS162)
Kirtan Soni(CE151)(19CEUON138)

B. Tech CE, Semester VI
Subject: System Design Practice

Guided By:
Prof. Apurva A Mehta
Assistant Professor
Dept. of Comp. Engg.

Department of Computer Engineering
Faculty of Technology, Dharmsinh Desai University
College Road,
Nadiad-387001

Faculty of Technology
Department of Computer Engineering
Dharmsinh Desai University

CERTIFICATE

This is to certify that the practical/term work carried out in the subject of **Software Design Project** and recorded in this journal is the bonafide work of

Kushal Shah, ID :19CEUOS160
Chintan Solanki, ID:19CEUBS162
Kirtan Soni, ID:19CEUON138

of B.Tech semester VI in the brach of Computer Engineering during the academic year 2021-2022.

Prof. Apurva A Mehta
Assistant Professor,
Dept. of Computer Engg.,
Faculty of Technology,
Dharmsinh Desai University,
Nadiad

Dr. C.K. Bhensdadia
Head,
Dept. of Computer Engg.,
Faculty of Technology,
Dharmsinh Desai University,
Nadiad

Acknowledgement

It is indeed a great pleasure to express our thanks and gratitude to all those who helped us during this project. This project would not have been materialized without the help from many who asked us good questions and rescued from various crisis.

Theoretical knowledge is of no importance if one doesn't know the way of its implementation. We are thankful to our institute that provided us an opportunity to apply our theoretical knowledge through the project. We feel obliged in submitting this project as part of our curriculum.

We would like to take the opportunity to express our humble gratitude to our guide **Prof. Apurva A Mehta**, under whom we undertook our project. His constant guidance and willingness to share his vast knowledge made us enhance our knowledge and helped us to complete the assigned tasks to perfection.
Without his effort, support and an astonishing testing ability this project may not have succeeded.

With Sincere Regards,
Chintan
Kirtan
Kushal

Table of Contents

- Certificate
- Acknowledgement
- Abstract

1 Introduction 6

2 Software Requirement Specification 8

3 System Design 10

3.1 Use Case Diagram 10

3.2 State Diagram 11

3.3 Activity Diagram 12

3.4 Sequence Diagram 13

4 Implementation 14

5 Screenshots 27

6 Conclusion 31

7 Limitations and Future Extension 32

8 Bibliography 33

Abstract

We have developed a text prediction project based on principles of Machine Learning. As we know every day with increase in digital devices the literature as well as soft documents and even text messages have become part of the human lives. As this is a machine learning project once the model is appropriately trained then the functionality of this project can be translated into infinite number of test cases starting from predicting upcoming words while sending a message to any important document being typed and generated on a computer machine. But for demonstration purposes we have created a simple User Interface that allows users/testers to actually test the output of the provided input. To establish a connection between the model designed in python languageand the front-end User Interface we have used the Flask Framework.

Main goal of this project is to predict upcoming words in a sentence as accurately as possible. while the input provided from user may or may not be grammatically perfect or having proper structure of sentence.

1. Introduction

This project will suggest upcoming words in a sentence while the user is still providing the model with new words. The prediction is done by considering the previous 3 words entered by the user. This sometimes allows the model to get very basic idea about the context of the sentence hence the predictions can be improved.

The project will be trained on certain books which possess accurate grammar and has proper language utilization. rather than some regional/rough words that can damage the accuracy of the project. The system will perform these tasks when provided input by the user:

1. Send window of N words to the model
2. Model will calculate probability of possible upcoming words
3. compare the probabilities calculated and filter out the highest one
4. The word with highest probability will be sent to the user interface tobe presented to the user.

1.1 Scope

This project is not limited to any specific boundaries. There are infinite number of possible use cases. Basically, this project can be utilized where the language English is being used.

1.2 Technology Used

- NLTK: NLTK is a leading platform for building Python programs to work With human language data.

- Flask: Flask is a micro web framework written in Python. It is classified as a microframework because it does not require particular tools or libraries.

- Jupyter Notebook: The Jupyter Notebook is the original web application for creating and sharing computational documents. It offers a simple, streamlined, document--centric experience.

1.3 Python Libraries Used

- TensorFlow: used to build and train models
- NumPy: used for data structures
- Pickle: to store dictionary index data.

1.4 Languages Used

- Python
- HTML/CSS

2. Software Requirements Specification

2.1 Product Scope

This system is designed to perform in various scenarios where text predictions can be utilized. scope of this system is global and open for all the users. Some of these uses maybe as follows but is not limited to: Conversational applications, Documentations, etc.

2.2 Types of Users

Here, there is only one end user who is going to use thisapplication.
so, there are no divisions of user required as this project does not need any kind of administration.

2.3 System Functional Requirements

1) Input from user:

User will see a textbox (HTML text area) to enter the text which is supposed to be processed to get the predictions from the back-end server. This data will Be treated as string by the python program.

2) Live Fetch and post data to backend:

This feature allows user to get processed output from back-end even when the data is still being input to system by the user. This works in a manner where as soon as user hits spacebar after entering some English words in the text area the data will be sent to the backend server and processed output will be displayed on the screen. now after this if the user keeps on typing then as soon as he enters spacebar again then this updated data will be sent to processing again and user will see a new prediction based on the updated input.

3) Predictions:

The data will be sent back to the server where the NLTK is implemented then the string data will be split into words then tokenization of these words will happen and after calculating probabilities the one word with the highest probability will be displayed to the user.

2.4 Non-functional Requirements

1) Performance:

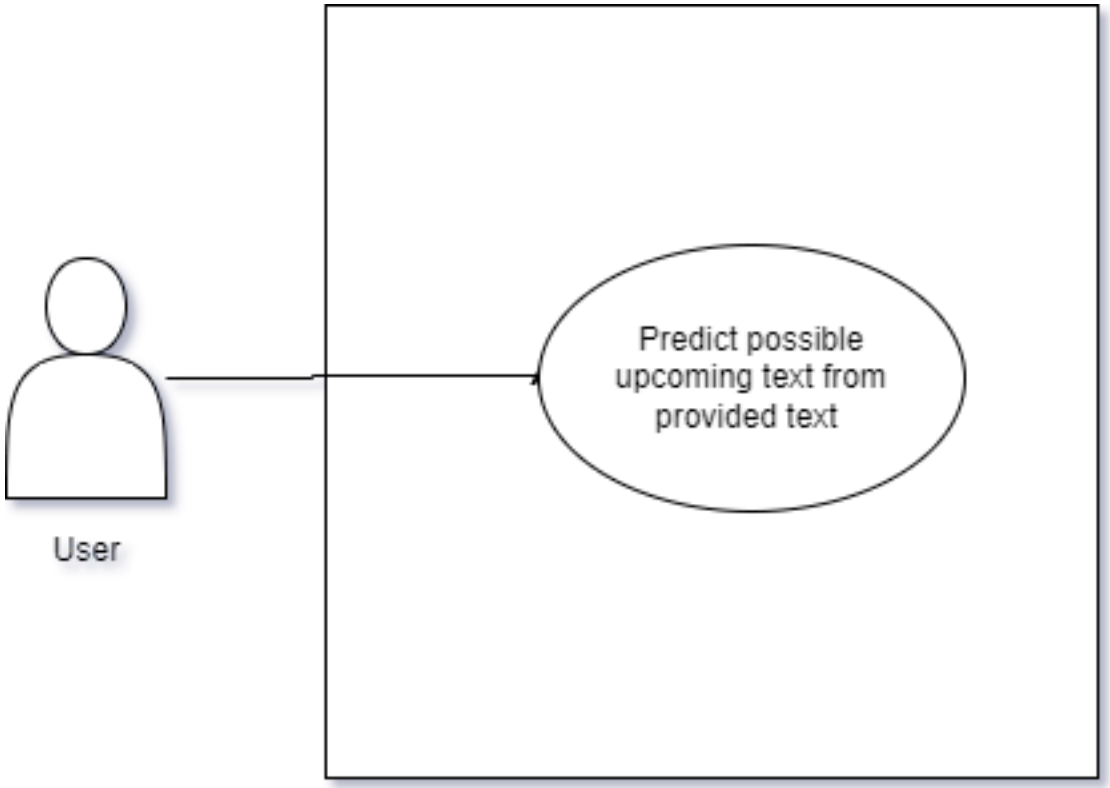
The application should run very efficiently. it must be user friendly in nature and the prediction should be displayed to user as fast as possible after user hits the spacebar after writing some words.

2) Reliability:

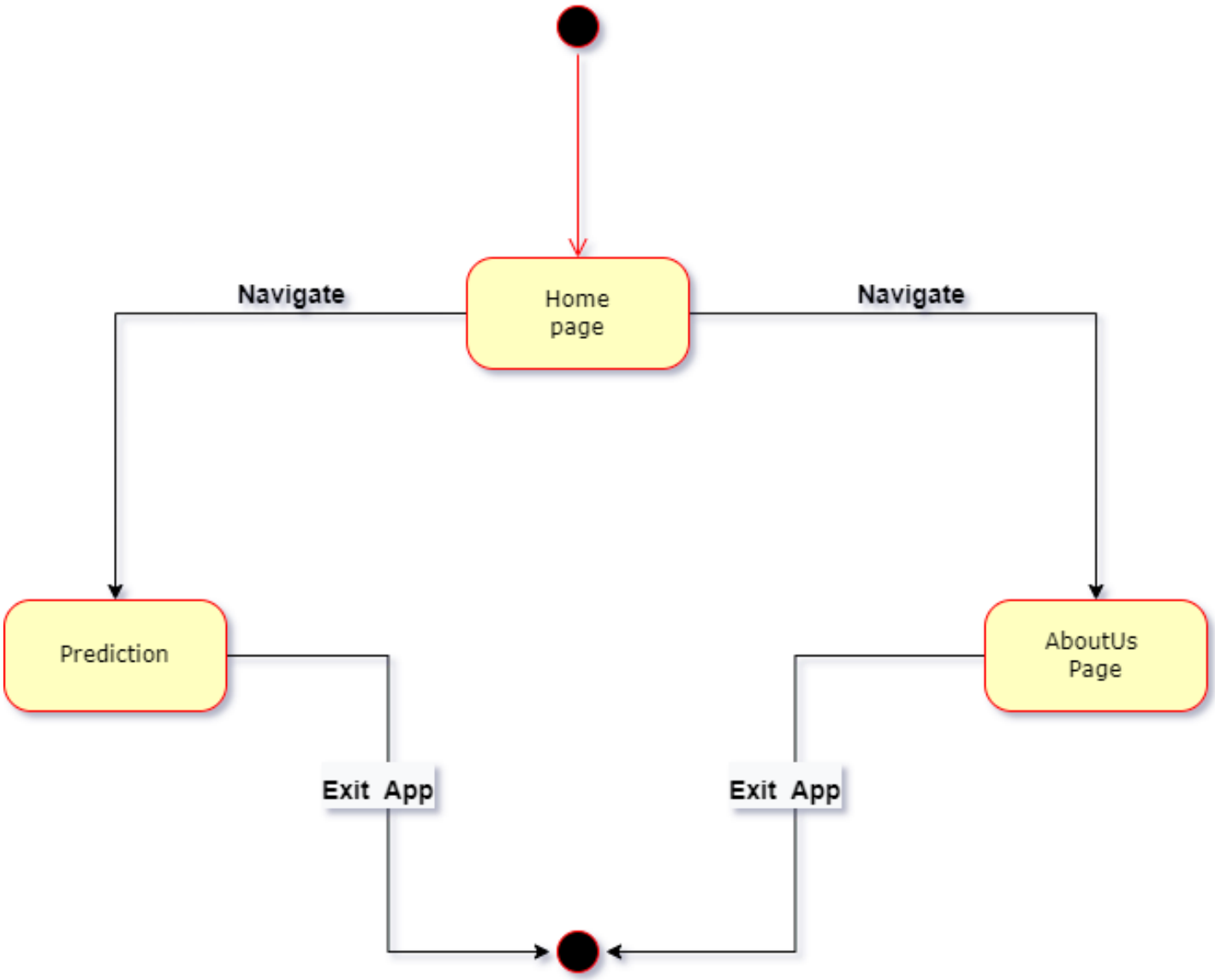
The predictions returned by the program should be in alignment to the text entered by the user. predictions should be as accurate as possible and relatable to the user's context and phrases entered by the user in the input sentence.

3. Design

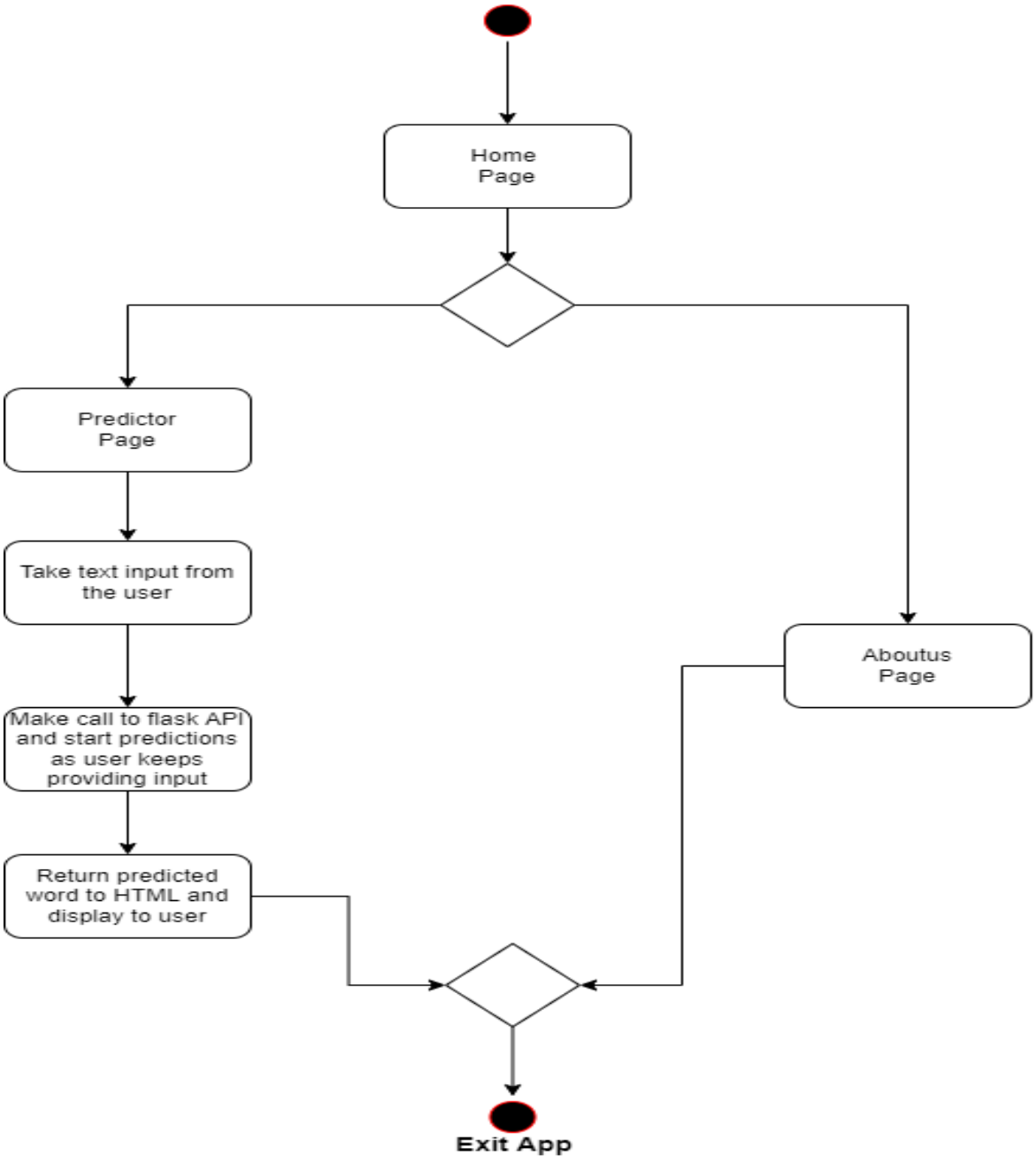
3.1 Use case diagram



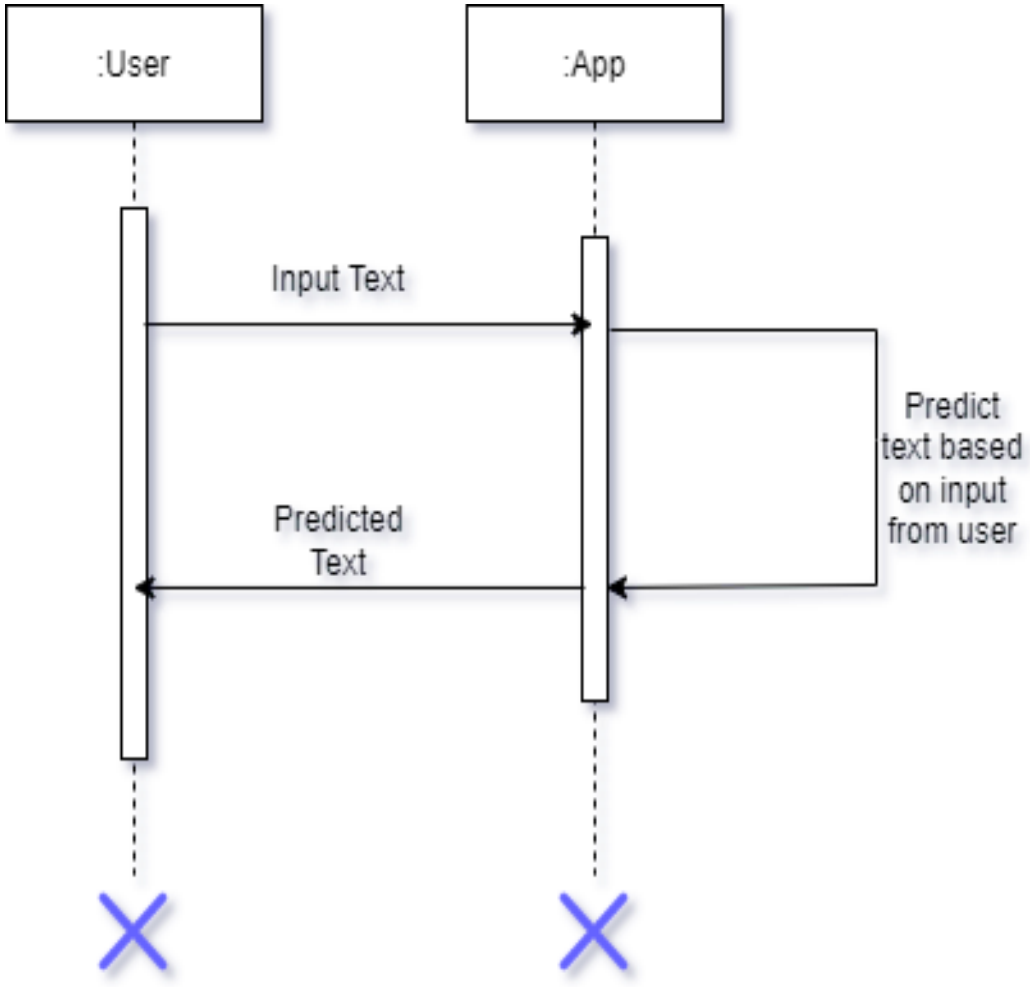
3.2 State Diagram



3.3 Activity Diagram



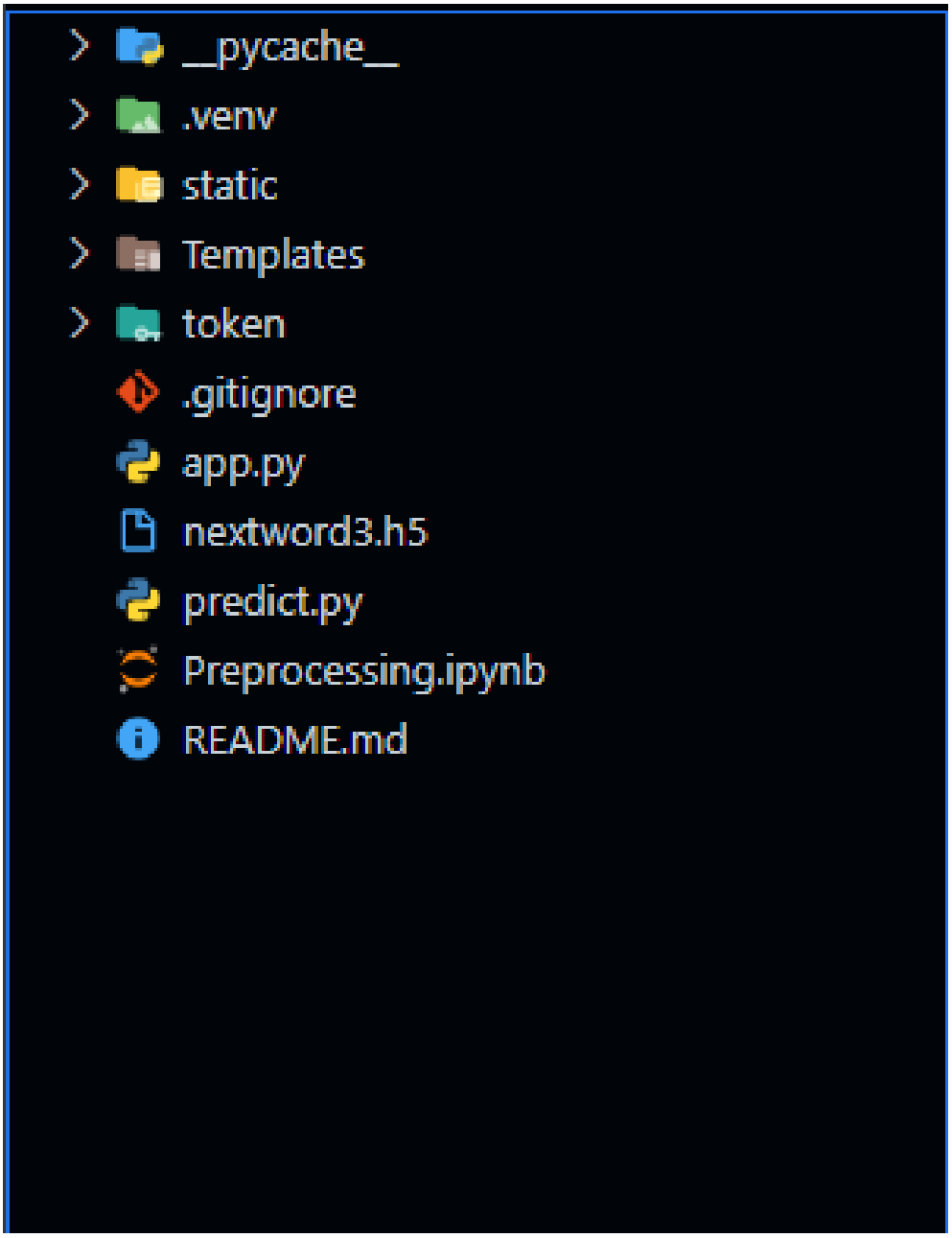
3.4 Sequence Diagram



4. implementation details

4.1 Front end implementation details

Front end is designed by using the python designed micro-framework Flask. in our project the Flask folder structure is as per follows:



here,

.venv contains all the files that are required for the virtual environment setup that is needed to run Flask applications.

static contains all the static files used such as .jpeg image files, .svg icon files and other formatting files such as CSS.

templates contain all the html files that will be rendered as per the request received by user and processed by the backend.

token contains the pickle tokenizer file used by backend.

The files mentioned above are those to design the front-end while **app.py, predict.py, Preprocessing.ipynb and nextword3.h5** are the files required to run the project calculations in back-end part.

4.2 Project Implementation Details:

Dataset:

In any machine learning project, the training dataset plays the most important role in improving the accuracy of the model in our project the we have tried to achieve as good predictions as we can by testing out our modelon different datasets.

Word predictions can be made accurate by using any dataset having grammatically correct sentences such as English books having pure English content or a dataset containing conversations between people can also be helpful.

The primary datasets which returned some good results and were utilized in the model training are listed below:

1. **Metamorphosis by the Gutenberg Project (Book)**
2. **Corpus ASCII (conversational dataset)**

before using any dataset, it requires to be preprocessed in order to remove any unnecessary data from the dataset. Then only the dataset can be fed to model to train here is a screenshot of preprocessing lines in which we have removed punctuation marks any digits and other information that can reduce the accuracy of the model while training.

Then any implementation specific to our project starts. because preprocessing of dataset in mandatory in all the machine learning projects. Next step is tokenization of data. In tokenization process, All the words in the dataset are assigned a digit, they together form a token which helps LSTM correlate between two different words. To perform tokenization TensorFlow library functions are used here are the screenshots containing code of tokenization and preprocessing:

(Loading Dataset)

```python
import os,glob
folder_path = './Dataset/CORPUS_TXT/'

for filename in glob.glob(os.path.join(folder_path, '*.txt')):
  with open(filename, 'r',encoding="UTF-8") as f:
    file = f.read()
    print (filename)
    print (len(file))
#file = open("./Dataset/dataset.txt","r",encoding="UTF-8")
    lines = []
    for i in file:
      lines.append(i)
    #print(lines)
```

CVE

```
Output exceeds the size limit. Open the full output data in a text editor
./Dataset/CORPUS_TXT\0001a.txt
9654
./Dataset/CORPUS_TXT\0001b.txt
8649
./Dataset/CORPUS_TXT\0001c.txt
10090
./Dataset/CORPUS_TXT\0001d.txt
13708
./Dataset/CORPUS_TXT\0001e.txt
14311
./Dataset/CORPUS_TXT\0002a.txt
```

Jupyter Server: local Cell 1 of 13 Go Live Prettier

(Preprocessing dataset)

+ Code + Markdown | Run All | Clear Outputs of All Cells | Outline ··· Select Kernel

```
from tensorflow.keras.preprocessing.text import Tokenizer
import pickle
tokenizer = Tokenizer()
tokenizer.fit_on_texts([newdata])
pickle.dump(tokenizer,open('./token/tokenizer1.pkl','wb'))
```

CVE

```
sequence_data = tokenizer.texts_to_sequences([newdata])[0]
#print(sequence_data)
vocab_size = len(tokenizer.word_index)+1
sequences = []
for i in range(3,len(sequence_data)):
  words = sequence_data[i-3:i+1]  #create window of 8 later on
  sequences.append(words)
print(sequences)
text = []
print(vocab_size)
# for i in sequences:
#   text.append(tokenizer.sequences_to_texts([i])[0])
# print(text)
```

CVE

+ Code + Markdown

(Tokenization)

As we can see in the last screenshot the specified window is 3 which means that predictions will be done by considering last 3 words entered by the user. Then a vocabulary is created by filtering unique words from the dataset and a 2D array is used for further data processing and training by the model. Find the screenshot below:

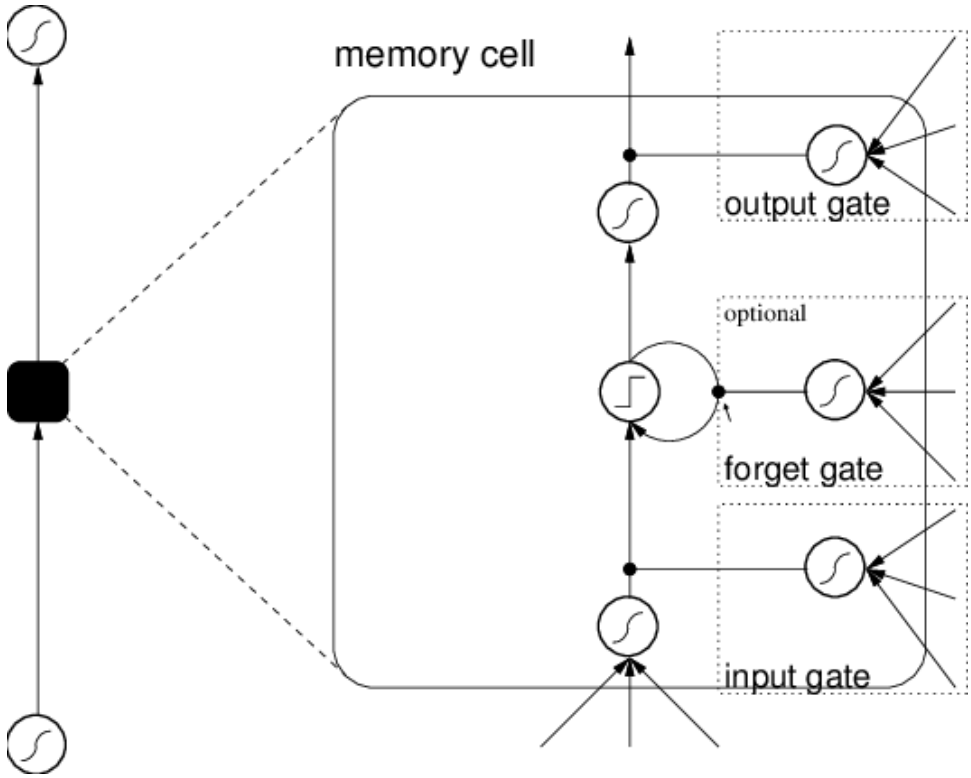
```
import numpy as np
from tensorflow.keras.utils import to_categorical
sequences = np.array(sequences)
x = []
y = []
for i in sequences:
    x.append(i[0:3])
    y.append(i[-1])
x = np.array(x)
y = np.array(y)
print(y)
y = to_categorical(y, num_classes=vocab_size)
print(x)
```

```
[ 12  95 920 ...  19 915 135]
[[ 54 142  57]
 [142  57  12]
 [ 57  12  95]
 ...
 [  3 762  32]
 [762  32  19]
 [ 32  19 915]]
```

Model used in the project:

After dataset a suitable model is very important for the project. In this project initially we have considered using Markov chain as well as recurrent neural network but then we realized that LSTM model fits our project very well so the implementation uses the LSTM model.

More about LSTM (Long Short-Term Memory):



Humans don't start their thinking from scratch every second. As you read this sentence, you understand each word based on your understanding of previous words. You don't throw everything away and start thinking from scratch again. Your thoughts have persistence.

Traditional neural networks can't do this, and it seems like a major shortcoming. For example, imagine you want to classify what kind of event is happening at every point in a movie. Itj's unclear how a traditional neural network could use its reasoning about previous events in the film to inform later ones.

Recurrent neural networks address this issue. They are networks with loops in them, allowing information to persist. LSTM are special type of the neural network described above they are capable to learn long term dependency of words in our case, LSTMs are explicitly designed to avoid the long-term dependency problem. Remembering information for long periods of time is practically their default behavior.

```
from tensorflow import keras
from tensorflow.keras.layers import Embedding,LSTM,Dense,Dropout
model = keras.Sequential()
model.add(Embedding(vocab_size, 10, input_length=3))
model.add(Dense(1000, activation="relu"))
model.add(Dropout(0.1))
model.add(LSTM(1000, return_sequences=True))
model.add(LSTM(1000))
model.add(Dense(1000, activation="relu"))
model.add(Dense(vocab_size, activation="softmax"))
```

Now the training starts where we specify the generated model file name and the loss functions calculations required to check the model is improved with every epoch in training.

We have trained model using 50 epochs but it can have any value. the in our case training started with loss value of 6.25868 (lesser the better) in first epoch and ended with loss value of 0.3432.

```
from tensorflow.keras.callbacks import ModelCheckpoint
from tensorflow.keras.callbacks import ReduceLROnPlateau
from tensorflow.keras.callbacks import TensorBoard

checkpoint = ModelCheckpoint("nextword3.h5", monitor='loss', verbose=1,
    save_best_only=True, mode='auto')

reduce = ReduceLROnPlateau(monitor='loss', factor=0.2, patience=3, min_lr=0.0001, verbose = 1)

logdir='logsnextword1'
tensorboard_Visualization = TensorBoard(log_dir=logdir)
```

```
from tensorflow.keras.optimizers import Adam
model.compile(loss="categorical_crossentropy", optimizer=Adam(lr=0.001))
model.fit(x, y, epochs=50, batch_size=64, callbacks=[checkpoint, reduce, tensorboard_Visualization])
```

And in the final snippet the function Predict_next_word is used to generate predictions from the tokenizer file which was stored using the pickle file(.pkl) which allows us to store raw datatype data to a binary file.

Predict_Next_word :

In this function we are using the tokenizer and models trained and we are creating the sequence of the text entered and then using our model to predict and return the predicted word.

```python
from tensorflow.keras.models import load_model
import numpy as np
import pickle

# Load the model and tokenizer

model = load_model('nextword3.h5')
tokenizer = pickle.load(open('./token/tokenizer1.pkl', 'rb'))

def Predict_Next_Words(model, tokenizer, text):
    """
        In this function we are using the tokenizer and models trained
        and we are creating the sequence of the text entered and then
        using our model to predict and return the the predicted word.

    """
    sequence = tokenizer.texts_to_sequences([text])[0]
    print(sequence)
    sequence = np.array([sequence])
    preds = model.predict(sequence)
    preds = np.argmax(preds,axis=1)
    #print(preds)
    predicted_word = ""
```

```python
    predicted_word = ""

    for key, value in tokenizer.word_index.items():
        if value == preds:
            predicted_word = key
            break

    print(predicted_word)
    return predicted_word
```

Here the return word (predicted_word) is the word with the highest probability and it is sent back the flask server to render in the html page generated to display predictions.

Handling the requests and serving responses from Flask:

predict.py > predict_next_words

```python
from tensorflow.keras.models import load_model
import numpy as np
import pickle

# Load the model and tokenizer

def predict_next_words(model, tokenizer, text):
    """
        In this function we are using the tokenizer and models trained
        and we are creating the sequence of the text entered and then
        using our model to predict and return the the predicted word.

    """
    sequence = tokenizer.texts_to_sequences([text])[0]
    sequence = np.array([sequence])

    try :
        preds = model.predict(sequence)
        preds = np.argmax(preds,axis=1)
        #print(preds)
        predicted_word = ""
    except :
        return "-"
    for key, value in tokenizer.word_index.items():
        if value == preds:
            predicted_word = key
            break

    #print(predicted_word)
    return predicted_word
```

Routing in Flask:

app.py > ...

```python
from cgitb import text
import os
from flask import Flask, send_file
from flask import request, jsonify
from flask import Flask, render_template, request
from predict import predict_next_words
import string
import pickle
from tensorflow.keras.models import load_model


with open('token/tokenizer1.pkl', 'rb') as handle:
    tokenizer = pickle.load(handle)
model = load_model('nextword3.h5')

photos = os.path.join('static', 'image')
svg = os.path.join('static', 'svgs')
app = Flask(__name__)
app.static_folder = 'static'
app.config['img_folder'] = photos
app.config['svg_folder'] = svg

txt = ""
req_dict = None
req_json = None


@app.route('/about_us')
def about_us():
    devimage = os.path.join(app.config['img_folder'], 'ak.jpg')
    fblogo = os.path.join(app.config['svg_folder'], 'facebook-brands.svg')
    linkedinlogo = os.path.join(app.config['svg_folder'], 'linkedin-brands.svg')
```

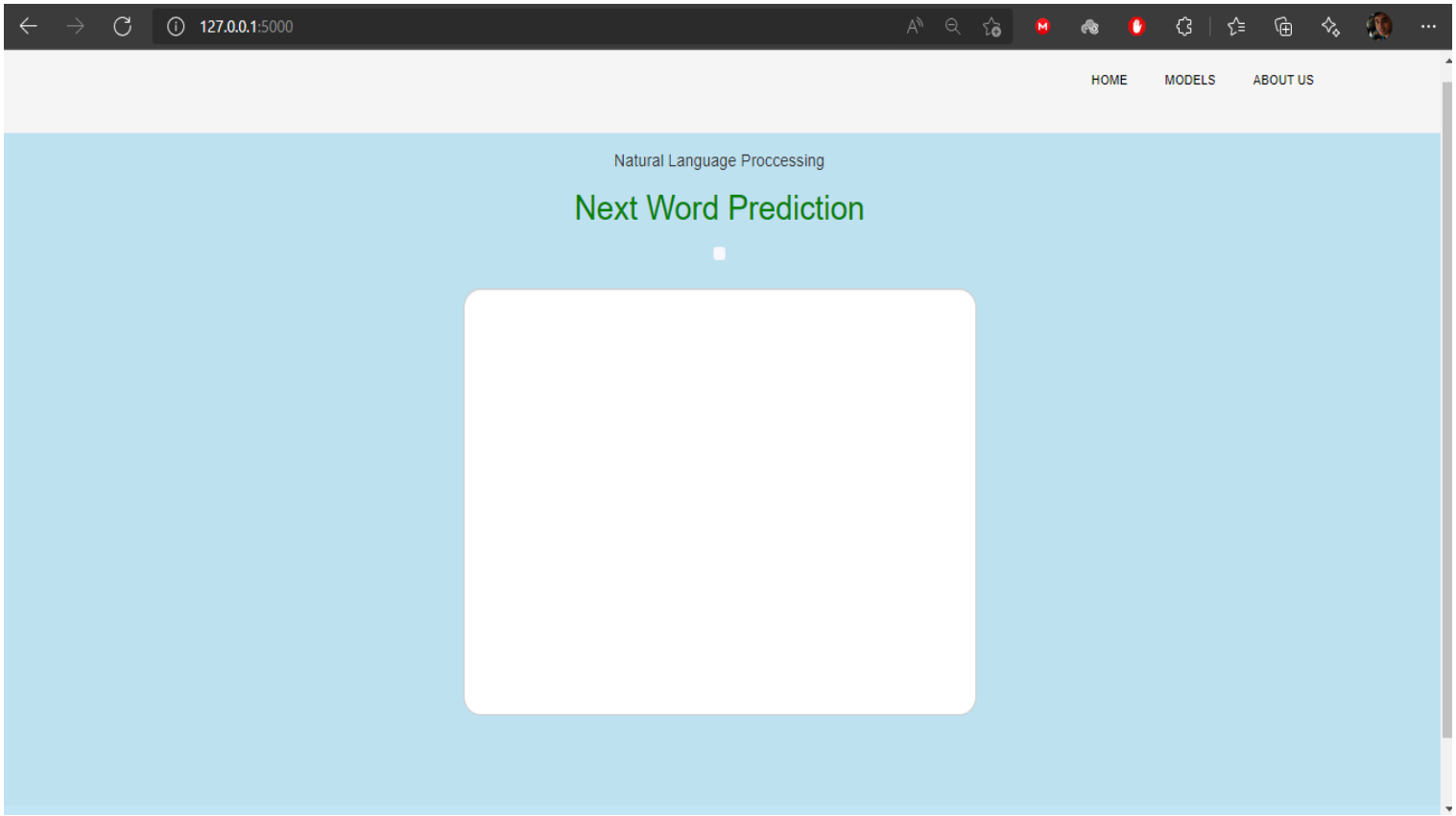
```python
    linkedinlogo = os.path.join(app.config['svg_folder'], 'linkedin-brands.svg')
    githublogo = os.path.join(app.config['svg_folder'], 'github-brands.svg')
    return render_template("about_us.html", devimage=devimage, facebook=fblogo, linkedin=linkedinlogo, gith

@app.route("/", methods=['GET', 'POST'])
def home():
    global req_dict, req_json
    '''if(request.method=='POST'):
        req_dict=dict(request.form)
        req_json=jsonify(req_dict)
        print(req_dict)
    return render_template("Home.html", prediction="Need more words")'''
    if request.method == "POST":
        global txt
        form = request.form
        predict = form['input_temp']
        txt = txt + predict + " "
        if predict == "":
            return render_template("Home.html", prediction="Need more words")
        predict = predict.split()[-3:]
        predict = predict_next_words(model, tokenizer, predict)
        print(predict)
        return render_template("Home.html", prediction=predict)
    else:
        return render_template("Home.html", prediction="Need more words")


if __name__ == "__main__":
    app.run(debug=True)
```

5. Deployment Screenshots:

(Predictions Page)



(About Us Page)

Our Team

KIRTAN SONI

Student

Roll No : 151
ID : 19CEUON138

CHINTAN SOLANKI

Student

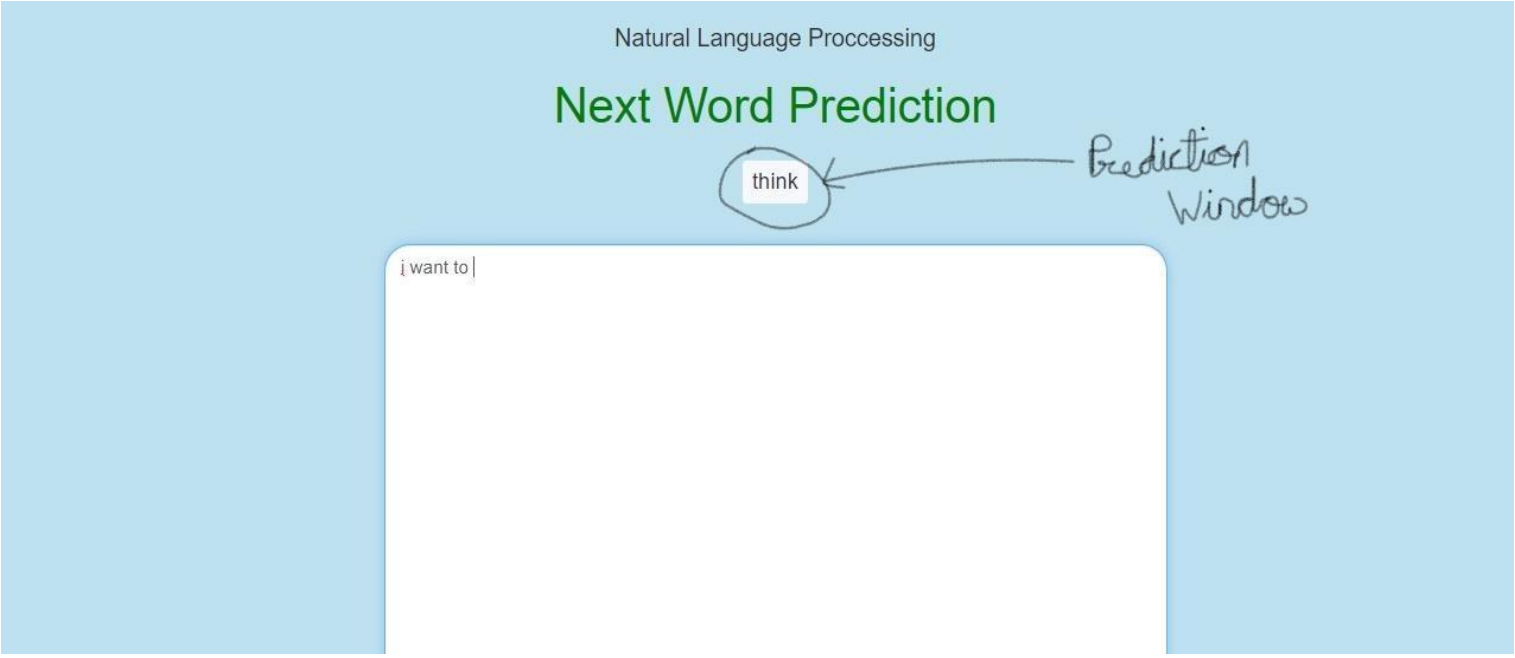
Roll No : 148
ID : 19CEUBS162

KUSHAL SHAH

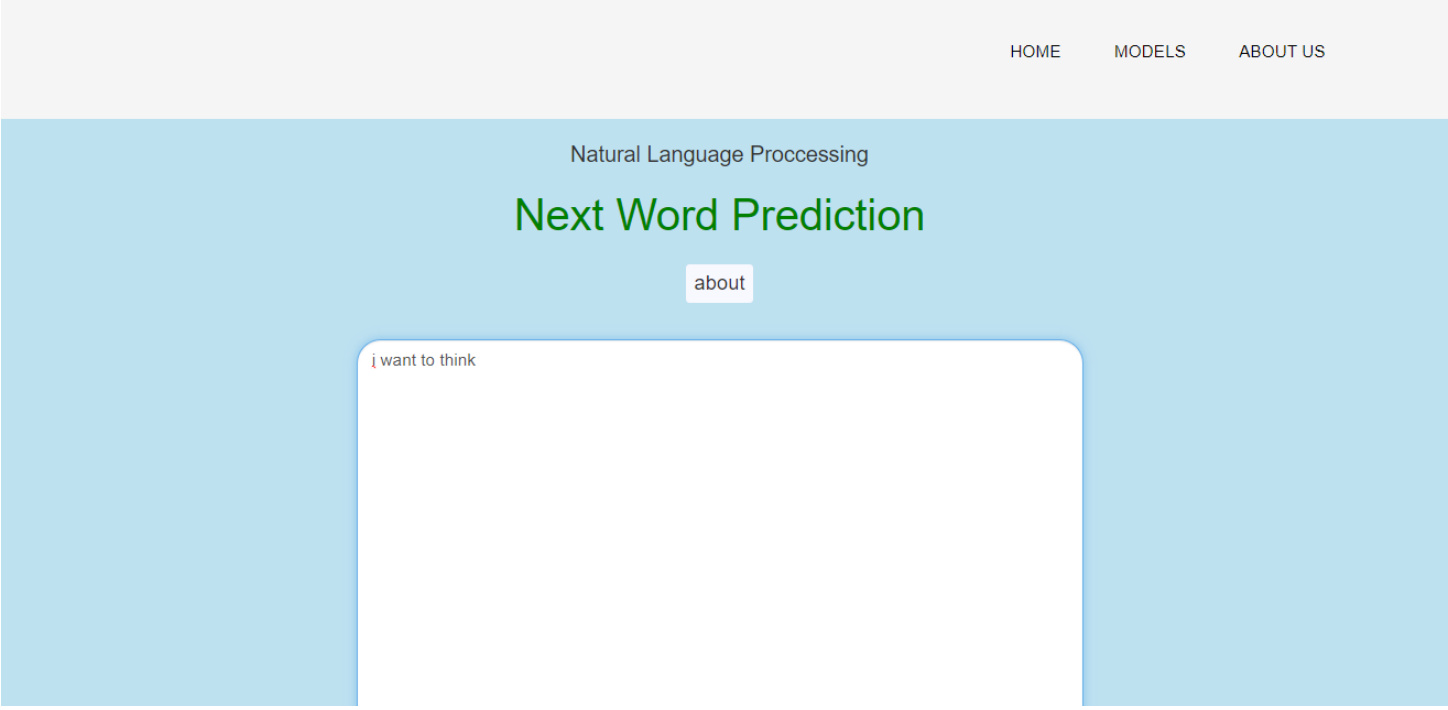
Student

Roll No : 142
ID : 19CEUOS160

As soon as user types in 3 words the prediction window starts displaying upcoming word with the highest probability like shown is screenshot below.



Now as user enters the suggested word or any other word and then hits spacebar so the word in prediction widow gets replaced by the new word as shown in screenshot below.


HOME
MODELS
ABOUT US
Natural Language Proccessing
Next Word Prediction
about
¡ want to think

6. Conclusion:

This project is aimed to develop a complete system that takes input from user and then process that input on a model trained on a dataset and provide the most relevant upcoming word predictions. As a machine learning project has no need to for any database connections or user administration, moreover there's no fixed use case.

As can be seen in the screenshots above there is no lag between input from user and the displaying of predictions

So, here we can conclude that our project is working well and predictions are good.

We have implemented these applications by writing code in python allthe webpages are designed in HTML with the help of CSS and Bootstrap library and the deployment server and framework used in development is Flask.

7. Limitations and future extensions:

Not just this one, but for any machine learning application there is no perfect dataset we have to find the suitable one but we cannot say that any other dataset cannot give us better results.

so, the search for fit dataset is an infinitely running process. The field is open for development so in future there can be another and better algorithms available making this one inferior to them.

Talking about extensions we are predicting upcoming word by considering previous 3 words from user input but mostly other famous predictions such as google and others use window of 8 previous words to predict. We can also Extend this project up to that target of 8.

Lack of enough computation power has been a problem through the development of this project because training the model even with 50 epochs can take up to 2 hours. On a better machine we can train the model with higher number of epochs to minimize the loss and perform better predictions.

8. Bibliography:

Datasets:

Free eBooks | Project Gutenberg
Dataset Search (google.com)
Find Open Datasets and Machine Learning Projects | Kaggle

Debugging:

Stack Overflow - Where Developers Learn, Share, & Build Careers

Version Control:

GitHub

Miscellaneous References:

Welcome to Flask — Flask Documentation (2.0.x) (palletsprojects.com)
Visual Studio Code - Code Editing. Redefined